Web Technology

BCS-502 · AKTU Exam Answers · Topper answer-sheet style

> **How to use these notes:** for code questions, write a **short program with comments** and then **explain the key tags or methods** in points. For theory questions, follow this order: definition, diagram, points, table, conclusion.

UNIT 1: WEB FOUNDATIONS, HTML & XML

Section A: 2-Mark Questions

Q1. Define HTTP and mention its three basic features. *[2 Marks]*

Ans. HTTP (HyperText Transfer Protocol) is an **application-layer, request-response protocol** for transferring web resources between a **browser (client)** and a **web server**. It runs over TCP on **port 80** (HTTPS uses 443).

1. **Connectionless:** the connection closes after each response.
2. **Stateless:** the server remembers nothing between requests, so cookies and sessions are used.
3. **Media independent:** any data type can be sent, using its **MIME type**.

Q2. Differentiate between HTML and XML. *[2 Marks]*

Ans.

HTML	XML
Displays data	**Stores and transports** data
Predefined tags	**User-defined** tags
Not case-sensitive	**Case-sensitive**
Closing tags optional, lenient	Closing tags **mandatory**, strict (well-formed)

Q3. Client-server vs. peer-to-peer paradigm. *[2 Marks]*

Ans.

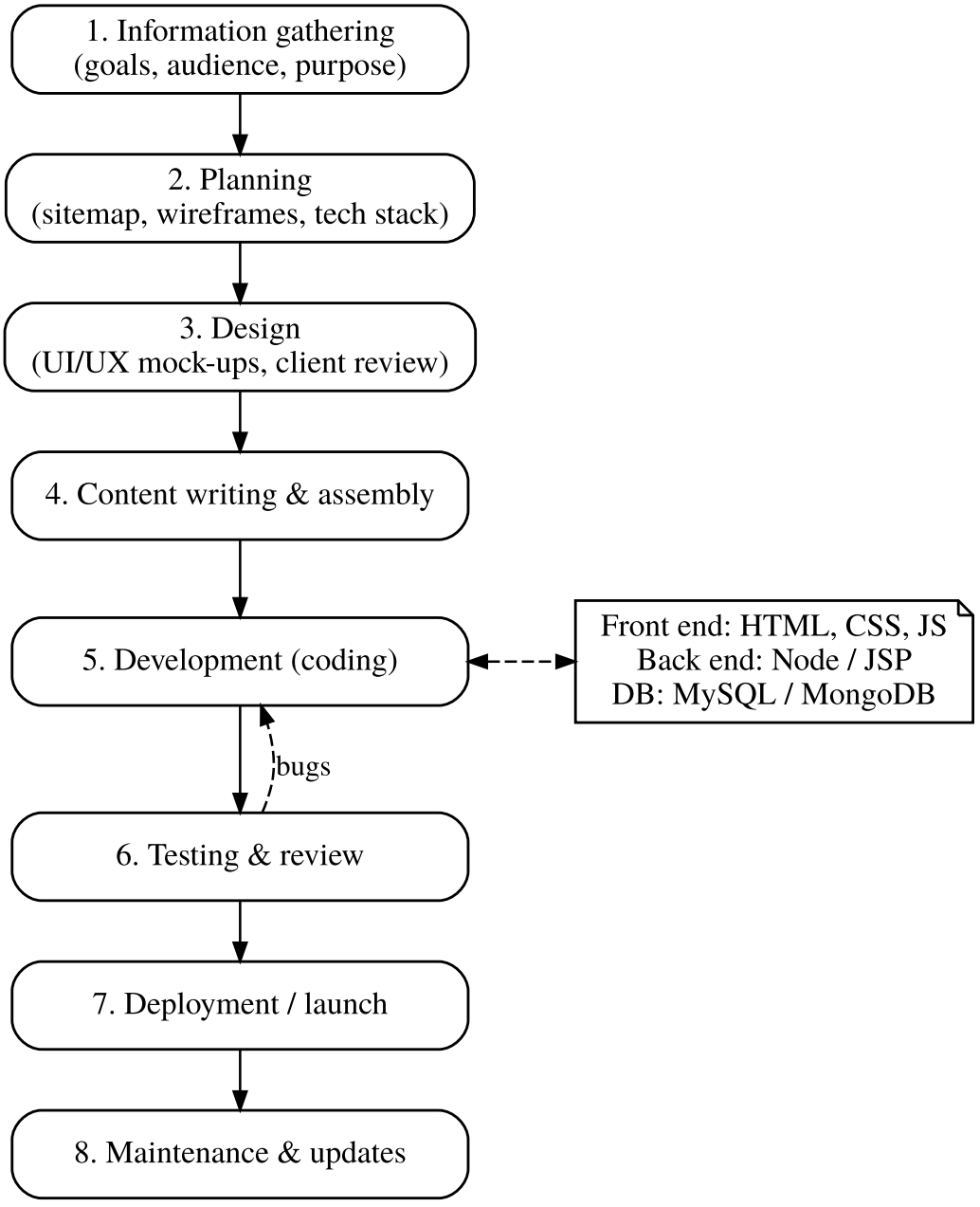
Client-Server	Peer-to-Peer
Dedicated server serves the clients	**Every node** is both client and server
Centralised; single point of failure	Decentralised; no single point of failure
Example: web browsing, email	Example: BitTorrent, blockchain

Section B: 7-Mark Questions

Q4. Explain web development strategies and the development process with a block diagram.

[7 Marks]

Ans. Web development means building, testing and maintaining websites and web applications. A **strategy** is the planned approach (goals, users, technology and method) used to deliver a site that is **usable, scalable and secure**.



Web development life cycle

Strategies:

- **Define goals and audience.**
- Choose the **architecture**: static site, dynamic site or **SPA**.
- **Mobile-first / responsive** design.
- **Methodology:** Waterfall or **Agile**.

- **Tech stack:** MERN or LAMP.
- Plan **SEO, accessibility and security** from the start.

Process (8 steps):

1. **Information gathering:** purpose and audience.
2. **Planning:** sitemap and wireframes.
3. **Design:** UI mock-ups.
4. **Content:** text, images, SEO.
5. **Development:** front end and back end.
6. **Testing:** browsers, forms, security.
7. **Deployment:** hosting and domain.
8. **Maintenance:** updates and monitoring.

Waterfall	Agile
Linear phases	Iterative sprints
Feedback at the end	Feedback every sprint
Changes costly	Changes easy

Conclusion: followed by every web team, for example when building an e-commerce site or a college ERP portal.

Q5. Write HTML for a user registration form using various input types. *[7 Marks]*

Ans. An **HTML form** collects user input. **action** gives the server URL and **method** (GET / POST) decides how the data is sent.

USER REGISTRATION FORM

Full Name [text]

Email [email]

Password [••••••••]

Date of Birth [dd-mm-yyyy (date)]

Gender (•) Male ( ) Female ( ) Other

Course [B.Tech CSE]

Skills [x] HTML [ ] CSS [x] JS

Photo [Choose File]

Address [ ]

[x] I agree to the terms

[Register] [Reset]

method="post" → action="/register" → server (Servlet / Node.js)

Layout of the registration form and where its data goes

```
<form action="/register" method="post" enctype="multipart/form-data">
  Name:     <input type="text" name="name" required><br>
  Email:    <input type="email" name="email" required><br>
  Password: <input type="password" name="pwd" minlength="8"><br>
  DOB:      <input type="date" name="dob"><br>
  Mobile:   <input type="tel" name="mob" pattern="[0-9]{10}"><br>
  Gender:   <input type="radio" name="g" value="M" checked> Male
            <input type="radio" name="g" value="F"> Female<br>
  Course:   <select name="course">
              <option>B.Tech CSE</option><option>B.Tech IT</option>
            </select><br>
  Skills:   <input type="checkbox" name="sk" value="html"> HTML
            <input type="checkbox" name="sk" value="js"> JS<br>
  Photo:    <input type="file" name="photo" accept="image/*"><br>
  Address:  <textarea name="addr" rows="3"></textarea><br>
  <input type="submit" value="Register"> <input type="reset">
</form>
```

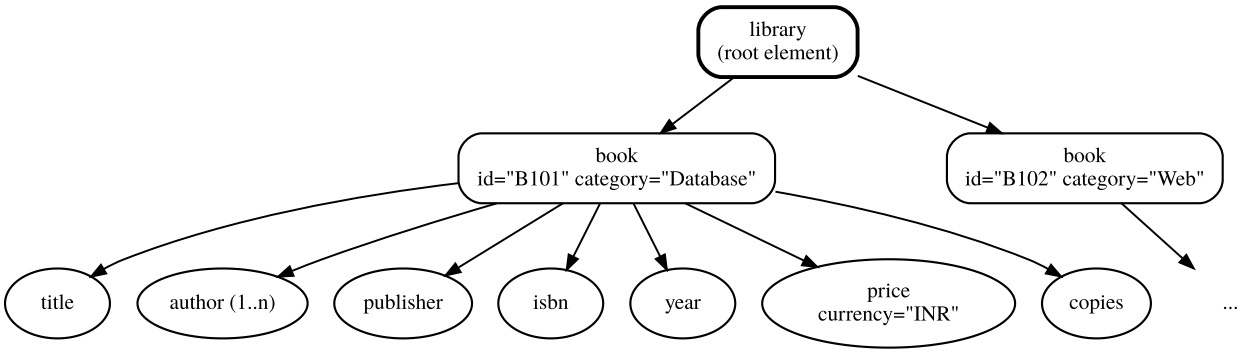
Key points:

1. **method=“post”** sends the data in the request body, which is secure for passwords.
2. **enctype=“multipart/form-data”** is required for **file upload**.
3. **name** is the key sent to the server, for example email=abc@x.com.
4. **required, pattern, minlength** give HTML5 validation.
5. **radio:** choose one option (same name). **checkbox:** choose many.

Conclusion: used on every sign-up page. Server-side validation is still required.

Q6. Write an XML file for library books and a DTD / XML Schema. *[7 Marks]*

Ans. XML stores data with **user-defined tags**. A **DTD or XSD** defines the legal structure, and a document that follows it is **valid**.



XML tree of library.xml, validated against a DTD or XSD

```
<?xml version="1.0"?>
<!DOCTYPE library [
  <!ELEMENT library (book+)>
  <!ELEMENT book (title, author+, isbn, year, price)>
  <!ELEMENT title (#PCDATA)>  <!ELEMENT author (#PCDATA)>
  <!ELEMENT isbn (#PCDATA)>   <!ELEMENT year (#PCDATA)>
  <!ELEMENT price (#PCDATA)>
  <!ATTLIST book id ID #REQUIRED category CDATA #IMPLIED>
]>
<library>
  <book id="B101" category="Database">
    <title>Database System Concepts</title>
    <author>Silberschatz</author> <author>Korth</author>
    <isbn>9780073523323</isbn> <year>2019</year> <price>850</price>
  </book>
</library>
```

XSD for one element (sample):

```
<xs:element name="book" maxOccurs="unbounded">
  <xs:complexType><xs:sequence>
    <xs:element name="title" type="xs:string"/>
    <xs:element name="year"  type="xs:gYear"/>
    <xs:element name="price" type="xs:decimal"/>
  </xs:sequence>
  <xs:attribute name="id" type="xs:ID" use="required"/></xs:complexType>
</xs:element>
```

Symbols:

- **+** = one or more, ***** = zero or more, **?** = optional.
- **#PCDATA** = text.
- **#REQUIRED / #IMPLIED** = mandatory / optional attribute.

DTD	XML Schema
Non-XML syntax	Written in XML
Text only	**44+ data types** (int, date)
No namespaces	Namespaces supported

Conclusion: XML with schemas is used in SOAP web services, configuration files and library systems.

UNIT 2: CASCADING STYLE SHEETS (CSS)

Section A: 2-Mark Questions

Q1. Three ways to integrate CSS; remove the underline from a link. *[2 Marks]*

Ans.

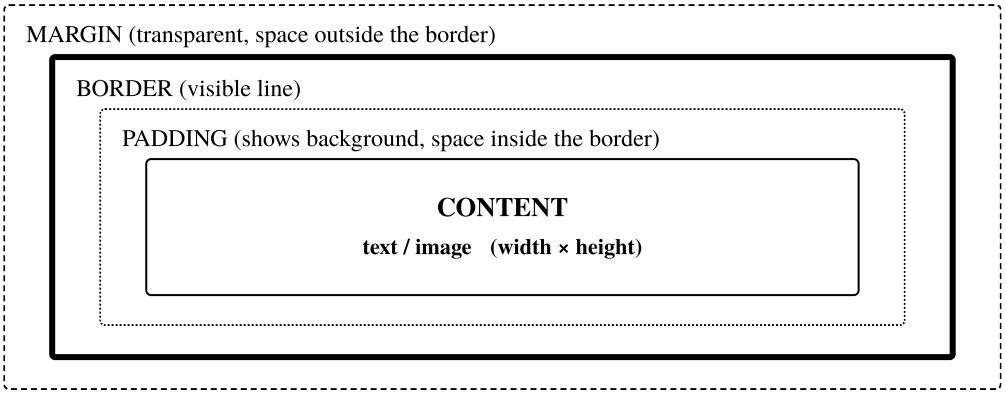
1. **Inline:** `<p style="color:blue">`. This has the **highest priority**.
2. **Internal:** a `<style>` tag in `<head>`, for one page.
3. **External:** `<link rel="stylesheet" href="style.css">`, for many pages.

To remove the underline: `a { text-decoration: none; }`

Section B: 7-Mark Questions

Q2. Illustrate the CSS Box Model with a diagram. *[7 Marks]*

Ans. The **CSS box model** says that **every element is a rectangular box** made of **content, padding, border and margin** (from the inside out).



CSS box model: margin → border → padding → content (outside to inside)

1. **Content:** text or image. Its size is set by `width` and `height`.
2. **Padding:** the space **inside the border**. It shows the background. Example: `padding: 10px;`
3. **Border:** the line around the padding. Example: `border: 2px solid black;`
4. **Margin:** the space **outside the border**. It is transparent, can be negative, and vertical margins **collapse**.

Size example: width 300, padding 20, border 5, margin 30:

$$\text{Total width} = 300 + 2(20) + 2(5) + 2(30) = 410 \text{ px}$$

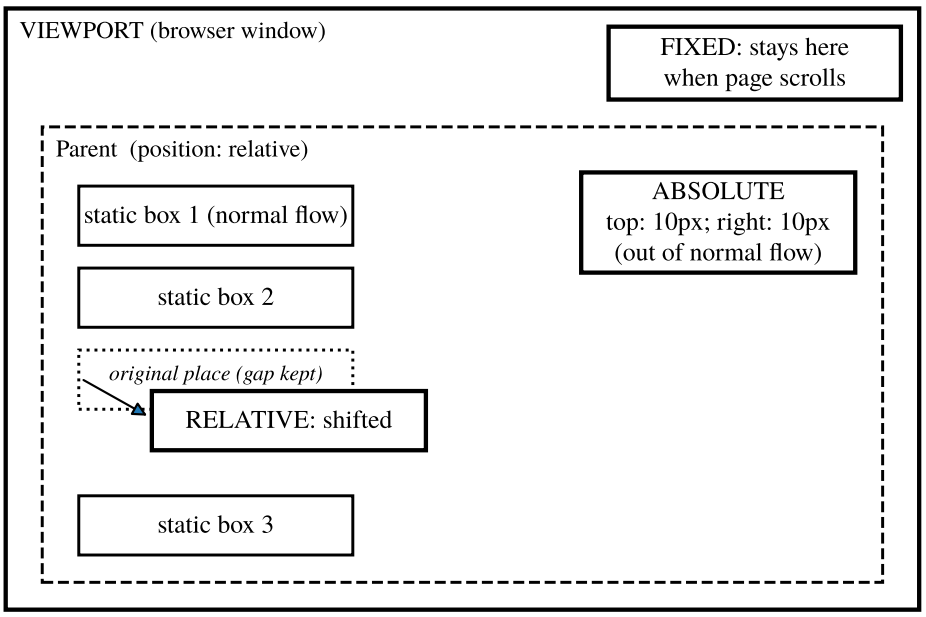
box-sizing: border-box makes `width` include the padding and the border.

Padding	Margin
Inside the border	Outside the border
Shows the background	Transparent
Cannot be negative	Can be negative

Conclusion: the box model is the base of every CSS layout (Flexbox, Grid).

Q3. Explain CSS positioning: static, relative, absolute, fixed. *[7 Marks]*

Ans. The **position** property decides how an element is placed. It works with **top, right, bottom, left** and **z-index**.



CSS positioning: static and relative stay in the normal flow; absolute is placed inside the nearest positioned parent; fixed is placed relative to the viewport

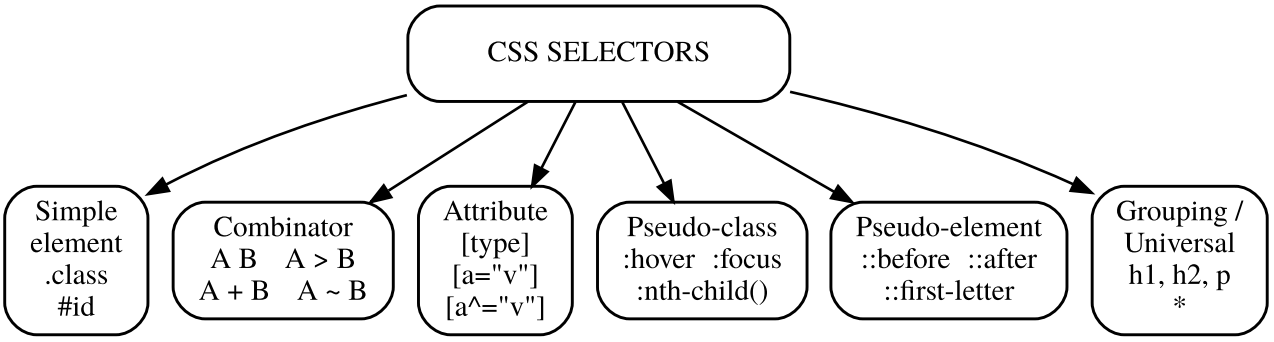
1. **static** (default): normal document flow. Offsets have no effect.
2. **relative:** moved **relative to its own normal position**. Its **original space is kept**. Example: `.box { position: relative; top: 20px; }`
3. **absolute: removed from the flow**. Placed relative to the **nearest positioned ancestor**. Example: a badge in a corner.
4. **fixed:** placed relative to the **viewport**. It **does not scroll**. Example: a sticky navbar.
5. **sticky:** acts as relative until a scroll point, then acts as fixed.

Value	In normal flow?	Placed relative to	Scrolls?
static	Yes	Normal flow	Yes
relative	Yes (gap kept)	Itself	Yes
absolute	**No**	Positioned ancestor	Yes
fixed	**No**	**Viewport**	**No**

Conclusion: used for navigation bars, dropdowns, modals and badges.

Q4. Discuss CSS selectors and pseudo-classes with examples. *[7 Marks]*

Ans. A **selector** chooses the HTML elements to style. A **pseudo-class** (`:`) selects an element in a **special state or position**.



Types of CSS selectors

1. **Simple:**
   - `*` (universal)
   - `p` (element)
   - `.cls` (class)
   - `#id` (id)
   - `h1, h2` (group)
2. **Combinators:**
   - `div p` (descendant)
   - `ul > li` (child)
   - `h1 + p` (adjacent sibling)
   - `h1 ~ p` (general sibling)
3. **Attribute:**
   - `input[type="text"]`
   - `a[href^="https"]` (starts with)
   - `a[href$=".pdf"]` (ends with)
4. **Pseudo-classes:**
   - `a:hover { color: red; }`, `input:focus`, `li:first-child`, `tr:nth-child(even)`, `:not(.x)`.
   - Write the link states in **LVHA order**: `:link`, `:visited`, `:hover`, `:active`.
5. **Pseudo-elements** (`::`): `p::first-letter`, `h2::before`, `::after`.
6. **Specificity:** inline (1000) > id (100) > class / pseudo-class (10) > element (1).

Pseudo-class	Pseudo-element
`:hover` (single colon)	`::before` (double colon)
A **state** of an element	A **part** of an element

Conclusion: used for zebra tables, hover menus and form-focus effects.

UNIT 3: JAVASCRIPT, AJAX & NETWORKING

Section A: 2-Mark Questions

Q1. What is AJAX and its main advantage? *[2 Marks]*

Ans. AJAX (Asynchronous JavaScript and XML) uses **XMLHttpRequest / fetch()** to exchange data with the server **in the background**. **Main advantage: partial page update without reloading**, so pages are faster and smoother. Examples: Google suggestions, Gmail refresh.

Q2. Java vs. JavaScript; define a Datagram. *[2 Marks]*

Ans.

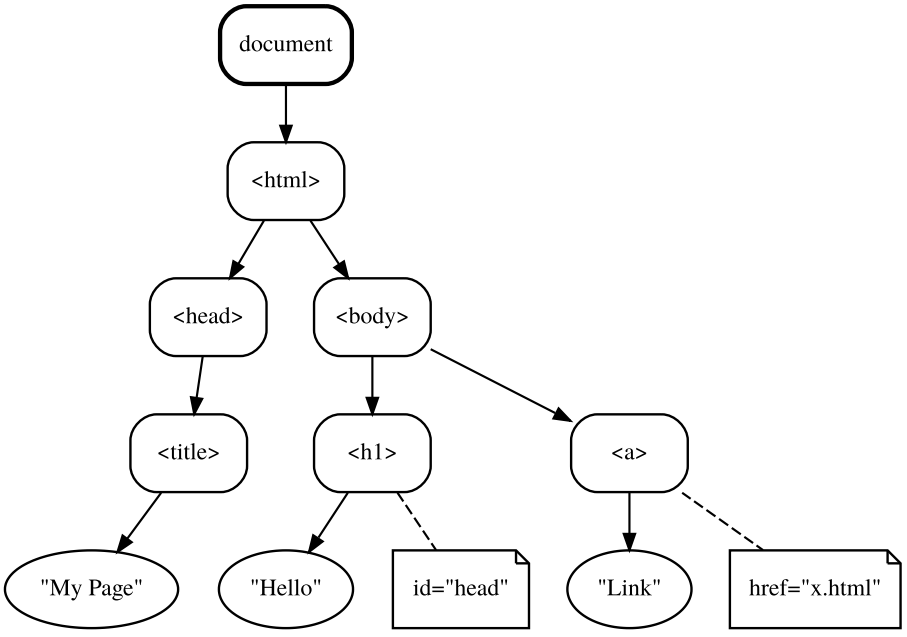
Java	JavaScript
Compiled to bytecode, runs on the **JVM**	Interpreted / JIT in the **browser**
Static typing, multi-threaded	**Dynamic** typing, single-threaded event loop

A **datagram** is an **independent, self-addressed packet** sent **without a connection** (UDP). It is **unreliable** (no guarantee of order or delivery), **fast** and of limited size. In Java: `DatagramSocket` and `DatagramPacket`.

Section B: 7-Mark Questions

Q3. Explain the JavaScript DOM. *[7 Marks]*

Ans. The **Document Object Model (DOM)** is a **W3C, language-independent interface** that represents an HTML document as a **tree of nodes**. JavaScript uses it to **read and change** content, structure and style **dynamically**.



DOM tree of the page: element, attribute and text nodes

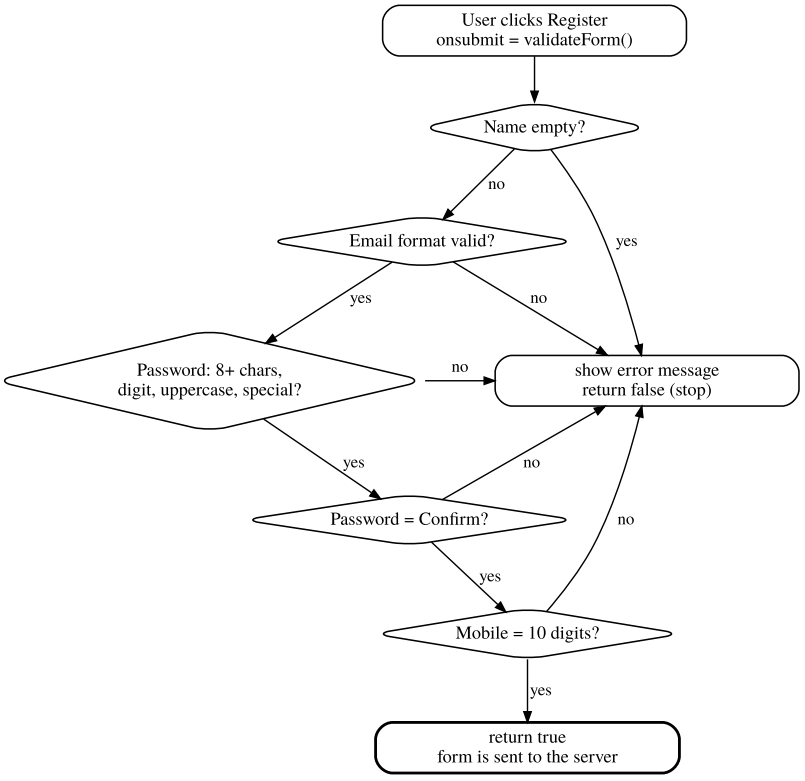
1. **Node types:** document, **element**, **attribute**, **text** and comment nodes.
2. **Select:** `getElementById()`, `getElementsByClassName()`, `querySelector()`, `querySelectorAll()`.
3. **Modify:** `el.innerHTML`, `el.textContent`, `el.setAttribute()`, `el.style.color = "blue"`.
4. **Create / remove:** `document.createElement("p")`, `appendChild()`, `removeChild()`.
5. **Traverse:** `parentNode`, `childNodes`, `firstChild`, `nextSibling`.
6. **Events:** `btn.addEventListener("click", fn)`. Events go through capturing, then the target, then **bubbling**.

DOM	BOM
Page content	Browser window
Root: `document`	Root: `window`
W3C standard	No standard

Conclusion: the DOM powers live validation, sliders, menus and AJAX updates.

Q4. Write a JavaScript program for form validation. *[7 Marks]*

Ans. Client-side validation checks input **in the browser before submission**, through the form's **onsubmit** event. Returning **false stops** the submission.



Client-side validation flow

```
        <form onsubmit="return validate()">
          Name: <input id="name"> Email: <input id="email">
          Password: <input type="password" id="pwd">
          Confirm: <input type="password" id="cpwd">
          <input type="submit" value="Register">
        </form>
        <script>
        function validate() {
          let n = document.getElementById("name").value.trim();
          let e = document.getElementById("email").value.trim();
          let p = document.getElementById("pwd").value;
          let c = document.getElementById("cpwd").value;
          if (n === "") { alert("Name is required"); return false; }          // empty
check
          if (!/^[\w.]+@[\w.]+\.[a-z]{2,}$/i.test(e)) {                        // email
format
              alert("Invalid email"); return false; }
          if (!/^(?=.*[A-Z])(?=.*\d)(?=.*[!@#$%]).{8,}$/.test(p)) {            // strong
password
              alert("Password: 8+ chars, 1 capital, 1 digit, 1 symbol"); return false; }
          if (p !== c) { alert("Passwords do not match"); return false; }     // confirm
          return true;                                                         // submit
form
        }
        </script>
```

Explanation:

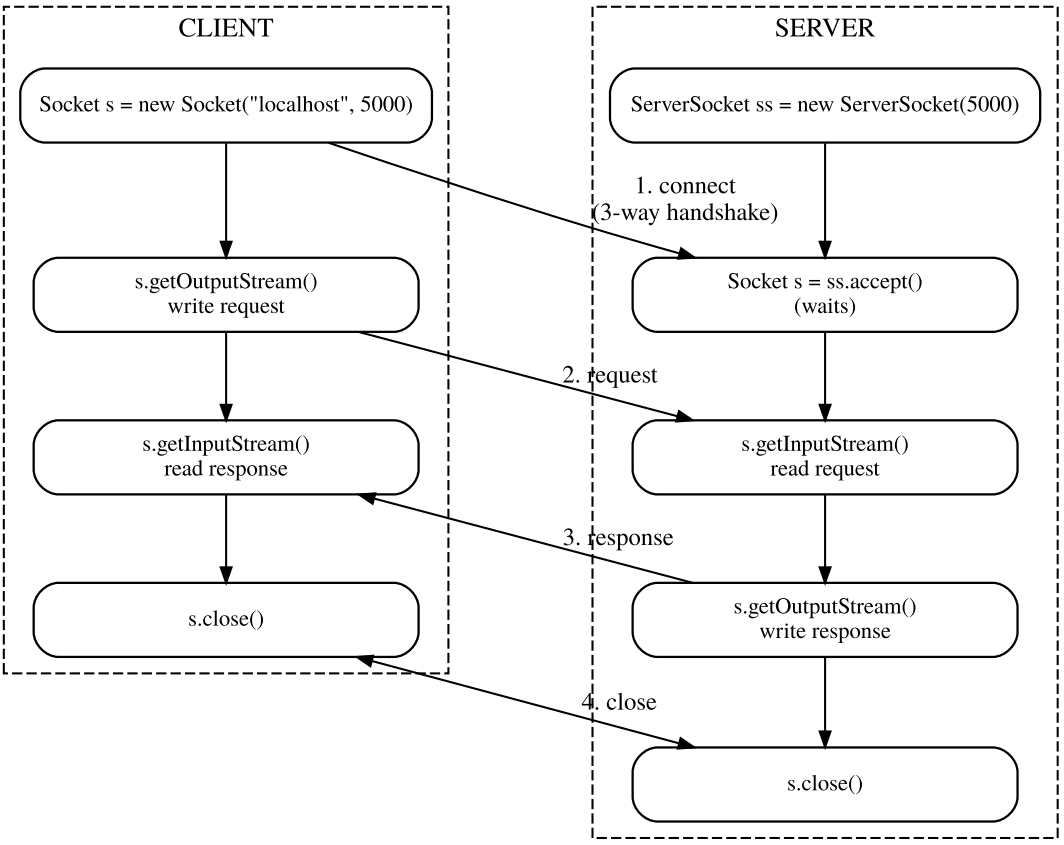
1. `.value.trim()` reads the input and removes extra spaces.
2. `RegExp.test()` checks the format.
3. `return false` cancels the submission.

Client-side	Server-side
Instant feedback	Needs a round trip
Can be bypassed	**Secure, mandatory**

Conclusion: used on login and sign-up forms, always together with server-side checks.

Q5. Explain TCP/IP client sockets with their constructors and methods. *[7 Marks]*

Ans. A **socket** is one **endpoint of a two-way link**, identified by an **IP address and a port**. A Java **TCP client socket (`java.net.Socket`)** gives a **reliable, connection-oriented stream**.



TCP client-server socket communication

Constructors:

1. `Socket(String host, int port)`: connects to a host and port.
2. `Socket(InetAddress addr, int port)`
3. `Socket(String host, int port, InetAddress localAddr, int localPort)`
4. `Socket()`: unconnected. Call `connect()` later.

Methods:

- `getInputStream()`, `getOutputStream()`
- `getInetAddress()`, `getPort()`, `getLocalPort()`
- `setSoTimeout()`, `isConnected()`, `close()`

```
        Socket s = new Socket("localhost", 5000);
        PrintWriter out = new PrintWriter(s.getOutputStream(), true);
        BufferedReader in = new BufferedReader(new InputStreamReader(s.getInputStream()));
        out.println("Hello Server");
        System.out.println("Server: " + in.readLine());
        s.close();
```

TCP Socket	UDP Datagram
Connection-oriented, reliable	Connectionless, unreliable
Byte stream	Packets
HTTP, FTP	DNS, video streaming

Conclusion: used by browsers, chat apps and JDBC drivers.

UNIT 4: JAVA BEANS & EJB

Section A: 2-Mark Questions

Q1. What is a Java Bean? What is introspection? *[2 Marks]*

Ans. A **Java Bean** is a **reusable software component** that follows these conventions:

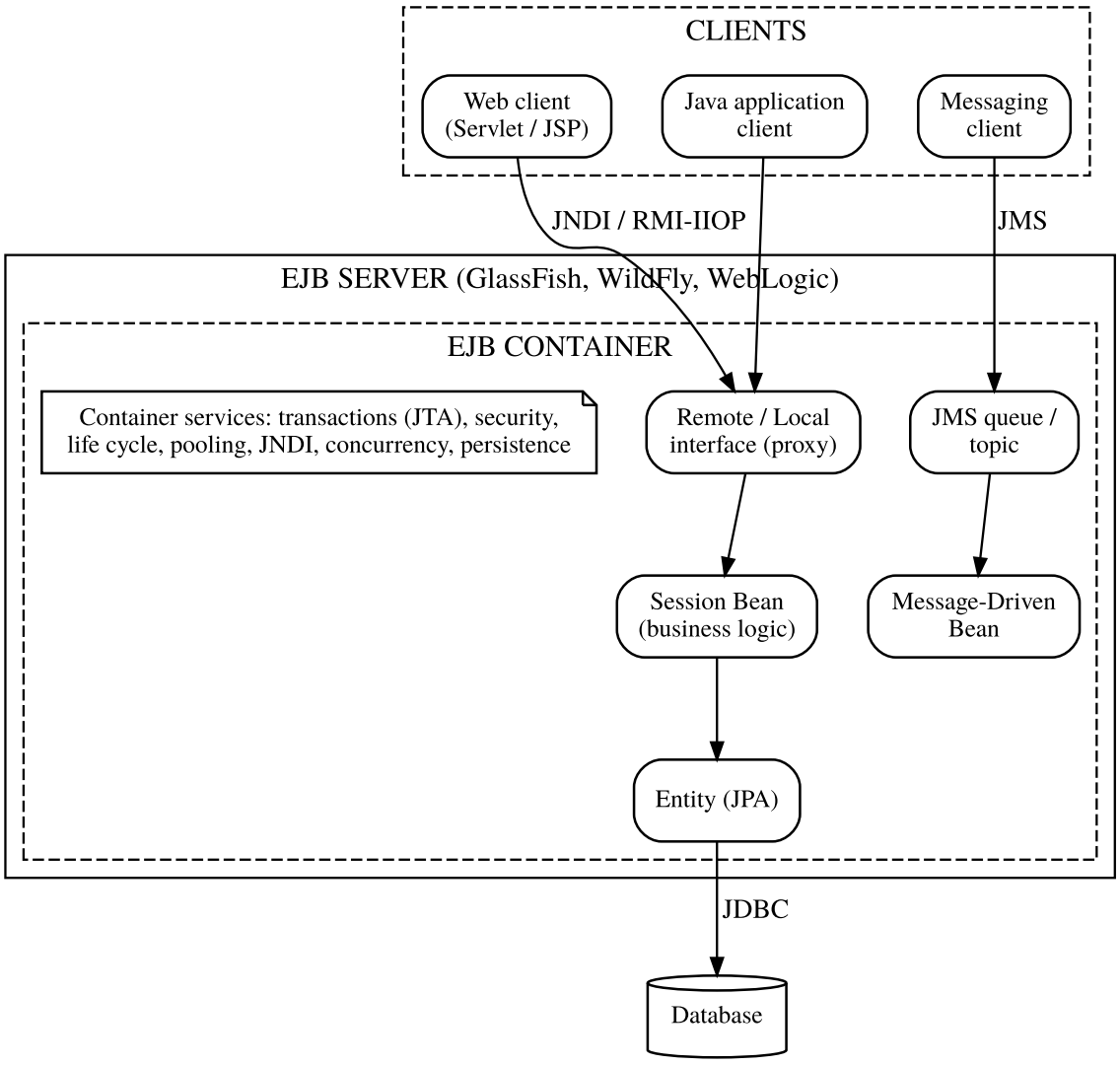
1. A **public no-arg constructor**.
2. **Private properties with getters and setters**.
3. It implements **Serializable**.

Introspection is the process by which builder tools **automatically discover a bean's properties, methods and events** at run time, using `java.beans.Introspector` (reflection plus naming conventions) or a `BeanInfo` class.

Section B: 7-Mark Questions

Q2. Explain the complete EJB architecture. *[7 Marks]*

Ans. Enterprise JavaBeans (EJB) is a **server-side component architecture** (Jakarta EE) for **distributed, transactional, secure** business applications. Beans run inside an **EJB container**, which gives them system services.



EJB architecture

1. **EJB server:** the application server (GlassFish, WildFly).
2. **EJB container:** manages the beans. **Every call passes through it.**
3. **Beans:**
   - **Session beans:** business logic.
   - **Message-driven beans:** asynchronous JMS messages.
   - **JPA entities:** persistence.
4. **Interfaces:** `@Remote` (other JVM, RMI-IIOP) and `@Local` (same JVM).
5. **JNDI:** clients look up beans by name.
6. **Container services:** transactions (JTA), security, pooling, life cycle, concurrency, persistence.

Request flow:

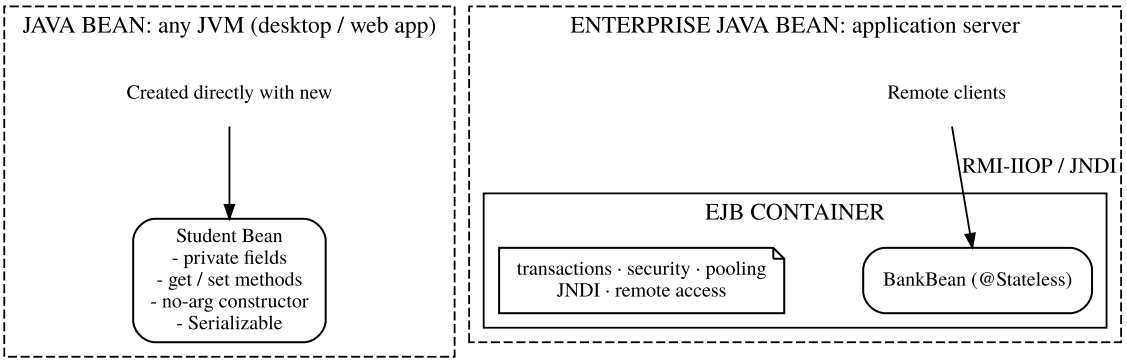
1. The client looks up the bean through JNDI.
2. The container returns a proxy.
3. The client calls a method on the proxy.
4. The container starts a transaction and checks security.
5. A pooled bean runs the method.

6. The container commits and returns the result.

Conclusion: EJB is used in banking and airline reservation systems that need distributed transactions.

Q3. Differentiate between a Java Bean and an EJB. *[7 Marks]*

Ans. A **Java Bean** is a simple reusable component. An **EJB** is a **server-side distributed business component** managed by a container.



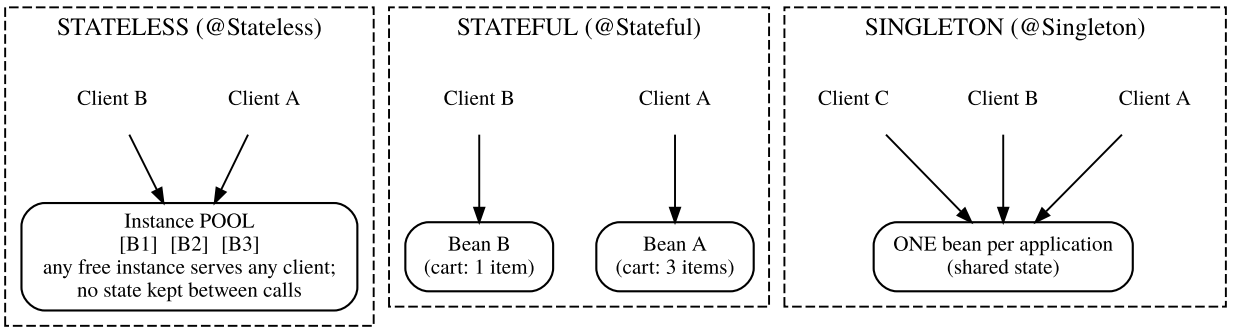
Where a Java Bean and an EJB run

Basis	Java Bean	EJB
Runs in	Any JVM	**EJB container** only
Created by	Programmer (`new`)	**Container**
Package	`java.beans`	`jakarta.ejb`
Remote access	No	Yes (RMI-IIOP)
Transactions / security	Coded manually	**Provided by the container**
Interfaces	None needed	`@Local` / `@Remote`
GUI use	Yes (builder tools)	No
Example	Swing button, Student DTO	Bank service bean
Complexity	Low	High

Conclusion: Java Beans are used for UI and data components. EJBs are used for enterprise business services.

Q4. Explain session beans: stateful vs. stateless. *[7 Marks]*

Ans. A **session bean** performs **business tasks for a client**. It is **not persistent**. There are three types: **Stateless, Stateful and Singleton**.



Session bean types



Life cycle of a stateful session bean (a stateless bean has only Does Not Exist and Ready)

1. **Stateless (`@Stateless`):**
   - Keeps **no client state** between calls.
   - Instances are **pooled and shared**, so it is highly scalable.
   - Example: a tax calculator.
2. **Stateful (`@Stateful`):**
   - Keeps **conversational state for one client**.
   - Can be **passivated** to disk when idle. Ends at `@Remove`.
   - Example: a **shopping cart**.
3. **Singleton (`@Singleton`): one instance per application**, shared by everyone. Example: a cache.

Stateless	Stateful
No conversational state	Keeps state per client
Shared pool	One bean per client
Very scalable	Uses more memory
No passivation	Passivation / activation

Conclusion: stateless beans for payment services, stateful beans for carts and booking wizards.

UNIT 5: SERVLETS, JSP, NODE.JS & MONGODB

Section A: 2-Mark Questions

Q1. What is Node.js? Purpose of HttpServletRequest? *[2 Marks]*

Ans.

- **Node.js** is a **JavaScript run time built on Chrome's V8 engine** for the **server side**. It is **single-threaded, event-driven and non-blocking**. **Use:** fast, scalable I/O apps such as REST APIs and

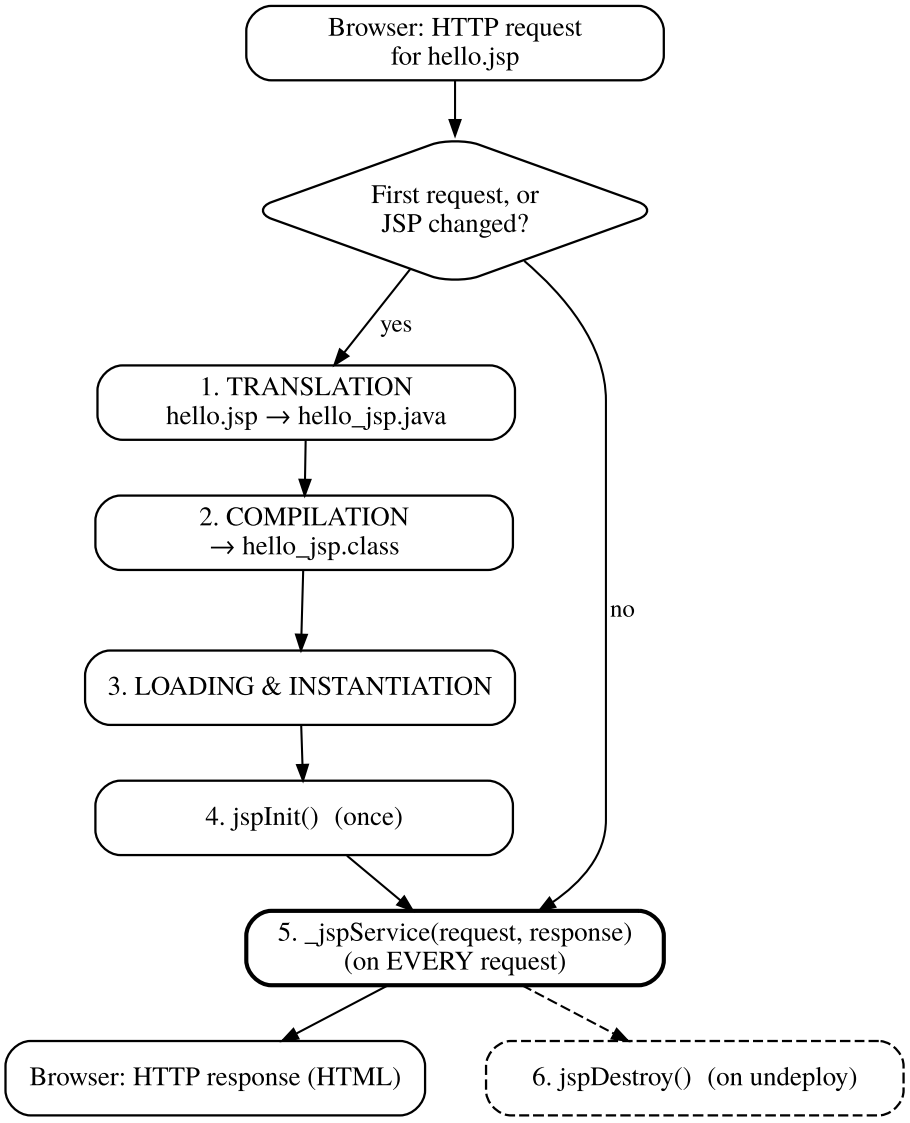
real-time chat.

- **HttpServletRequest** represents the **client's HTTP request**. It lets the servlet read parameters (`getParameter()`), headers, cookies, the session (`getSession()`) and the request method.

Section B: 7-Mark Questions

Q2. Explain the compilation of JSP into Servlets and its execution. *[7 Marks]*

Ans. A **JSP** page is not run directly. The **JSP container (Jasper in Tomcat) translates it into a servlet, compiles it and then executes it**.



JSP processing: translation and compilation happen only on the first request (or after the JSP changes)

1. **Translation:** hello.jsp becomes **hello_jsp.java**.
   - HTML becomes `out.write()`.
   - `<% %>` code goes into `_jspService()`.
   - `<%= %>` becomes `out.print()`.
2. **Compilation:** produces **hello_jsp.class**.
3. **Loading and instantiation:** one object is created.

4. **jspInit(): called once**.
5. **_jspService(req, res):** called for **every request**, in a new thread. Implicit objects are available: `request, response, out, session, application`.
6. **jspDestroy(): called once, on undeploy.

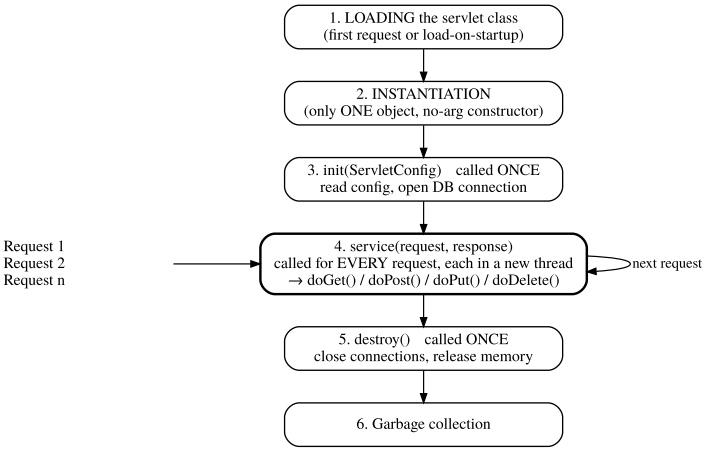
> **Remember:** only the **first request** is slow (translation and compilation). Later requests call `_jspService()` directly.

JSP	Servlet
Java inside HTML (view)	HTML inside Java (controller)
Auto-recompiles	Manual compile
Implicit objects	Get the objects manually

Conclusion: JSP is the **View** in MVC web applications running on Tomcat.

Q3. Explain the Servlet life cycle methods: init, service, destroy. *[7 Marks]*

Ans. A **servlet** is a server-side Java class that handles HTTP requests. Its **life cycle** is controlled by the **container** through `init()`, `service()` and `destroy()`.



Servlet life cycle

1. **Loading and instantiation:** the class is loaded on the first request (or at start-up), and **one object** is created.
2. **init(ServletConfig): called once**. Used for set-up, such as opening a DB connection.
3. **service(req, res): called for **every request**, each in a **new thread**. It dispatches to `doGet()` / `doPost()`.
4. **destroy(): called once**, before removal. Releases resources.
5. **Garbage collection.**

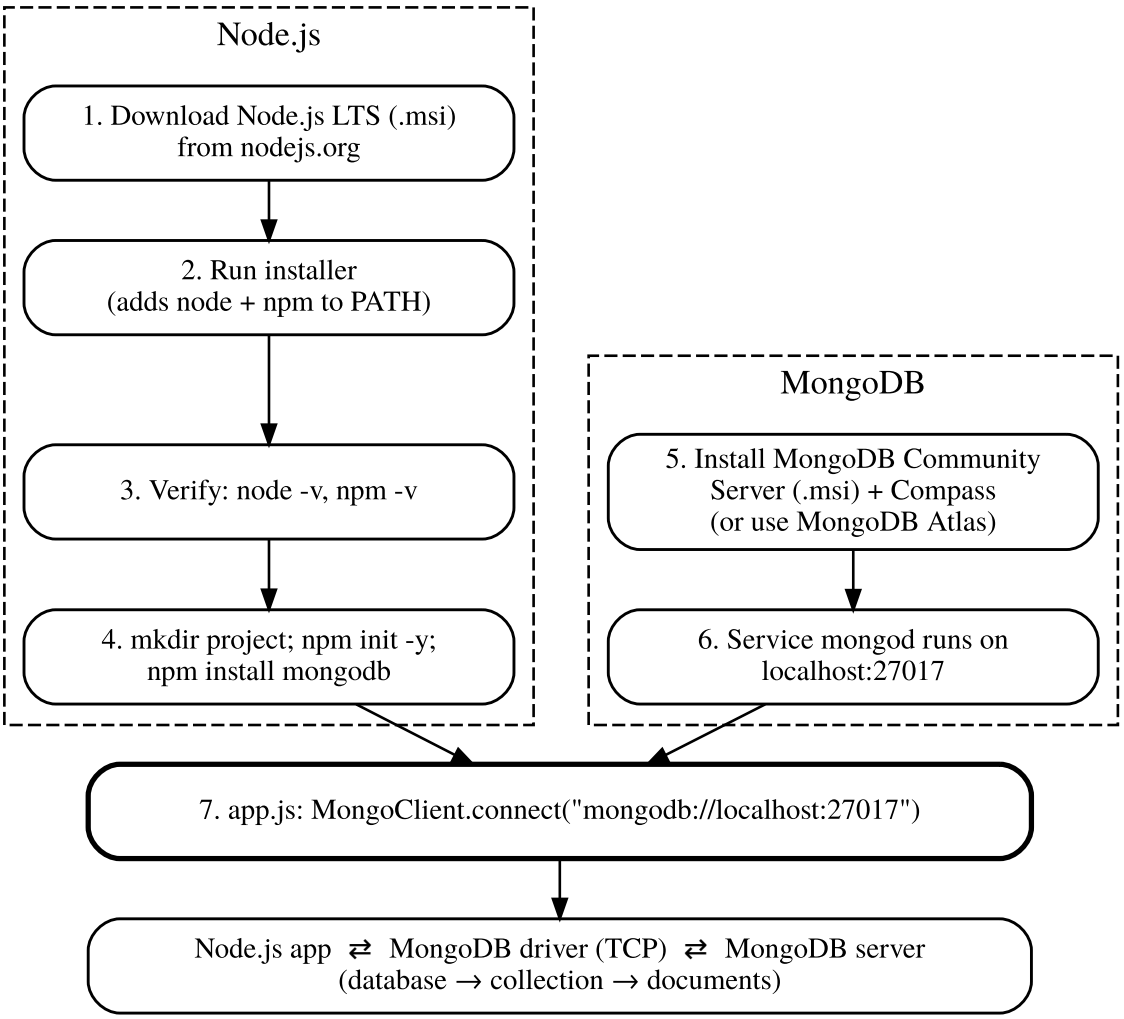
```
        public class Hello extends HttpServlet {
          public void init() { System.out.println("init once"); }
          protected void doGet(HttpServletRequest rq, HttpServletResponse rs) throws
IOException {
              rs.getWriter().println("<h2>Hello " + rq.getParameter("name") + "</h2>"); }
          public void destroy() { System.out.println("destroy once"); }
        }
```

init()	**service()**	**destroy()**
Once	Per request	Once
Set up	Process the request	Clean up

Conclusion: the basis of Spring MVC's DispatcherServlet and all Java web apps.

Q4. Steps to set up Node.js on Windows and connect to MongoDB. *[7 Marks]*

Ans. Node.js runs JavaScript on the server. **MongoDB** is a **NoSQL document database**. They are connected through the **mongodb driver** (or Mongoose).



Setting up Node.js and MongoDB on Windows

1. Download the **Node.js LTS (.msi)** from nodejs.org and install it (adds it to PATH).
2. Verify: `node -v` and `npm -v`.

3. `mkdir myapp && cd myapp`, then `npm init -y`, then `npm install mongodb`.
4. Install **MongoDB Community Server** (as a service). It runs on **localhost:27017**. (Or use MongoDB Atlas.)
5. Write `app.js` and run it with `node app.js`:

```
const { MongoClient } = require("mongodb");
const client = new MongoClient("mongodb://127.0.0.1:27017");
async function main() {
  await client.connect();                              // connect
  const db = client.db("collegeDB");                   // database
  await db.collection("students").insertOne({ roll: 101, name: "Harsh" });
  console.log(await db.collection("students").find().toArray());
  await client.close();
}
main();
```

MongoDB	MySQL
Documents (JSON)	Tables
Flexible schema	Fixed schema
Horizontal scaling	Mainly vertical scaling

Conclusion: Node.js with MongoDB forms the back end of the **MERN** stack.